

Hands-on ML/AI for Aquatic Remote Sensing

**Chintan B. Maniyar
Center for Geospatial Research
Department of Geography
University of Georgia
Athens, GA, USA**

Outline

- Part-1
 - Set-up Python environment and GitHub integration, Read Data
- Part-2
 - Exploratory data analysis, train-test-split, ML algorithms, effect of feature-engineering, metrics and visualization
- Part-3
 - NN, data transformations
- Part-4
 - Extending the model to satellite data and creating maps

What is GitHub and
why is it important?

GitHub: Open Science and Code

- GitHub is primarily a version control tool to manage, share and collaborate on code-based projects.
- In modern science, GitHub is a fundamental tool for “open source science”
- Ease in reproducibility
- In today’s age of **data science**, GitHub is like your **CV of coding skills**

Our GitHub repository for the day

- https://github.com/Chintan2108/SCOR_Workshop_2025

To-do [get to coding!]

- Create your GitHub account
- “Fork” the repository to your GitHub profile
 - Now you have a copy of this repository in your GitHub account
 - You will keep updating this with the exercise that you do
- “Clone” this repository on your local machine
- Create a personal access token
 - Profile → Settings → Developer settings → Personal Access Tokens
- Download Git for windows: <https://github.com/git-for-windows/git/releases/download/v2.52.0.windows.1/Git-2.52.0-64-bit.exe>
- Start downloading the satellite image – it may take long.

Basic Git Commands to Remember

- **git status**
 - What has changed in the repository?
 - Think: These are the things you need to pack in a box
- **git add .**
 - Add all the changes to your current commit
 - Think: You put all the things in the box
- **git commit -m "commit message here"**
 - Commit your changes with a message (finalize the changes)
 - Think: You packed your box, and put a message on it for the receiver
- **git push**
 - Send your commit to the Git Server (now you can see your changes online)
 - Think: You finally sent/couriered your box to destination (online GitHub)

Capstone: Chl-*a* modeling using ML Methods

- Data
 - *In situ* hyperspectral radiometry data collected from various cruises in the continental US and around the globe
 - Corresponding Chl-*a* concentrations derived analytically using methods viz wet lab analyses, HPLC, spectrophotometer etc.
 - Radiometry data has already been modified to match the spectral response of Sentinel-3 OLCI
- Goals
 1. Train a model using the *in situ* Chl-*a* data
 2. Evaluate the model using uncertainty metrics we discussed last time
 3. Apply trained model on the OLCI satellite imagery

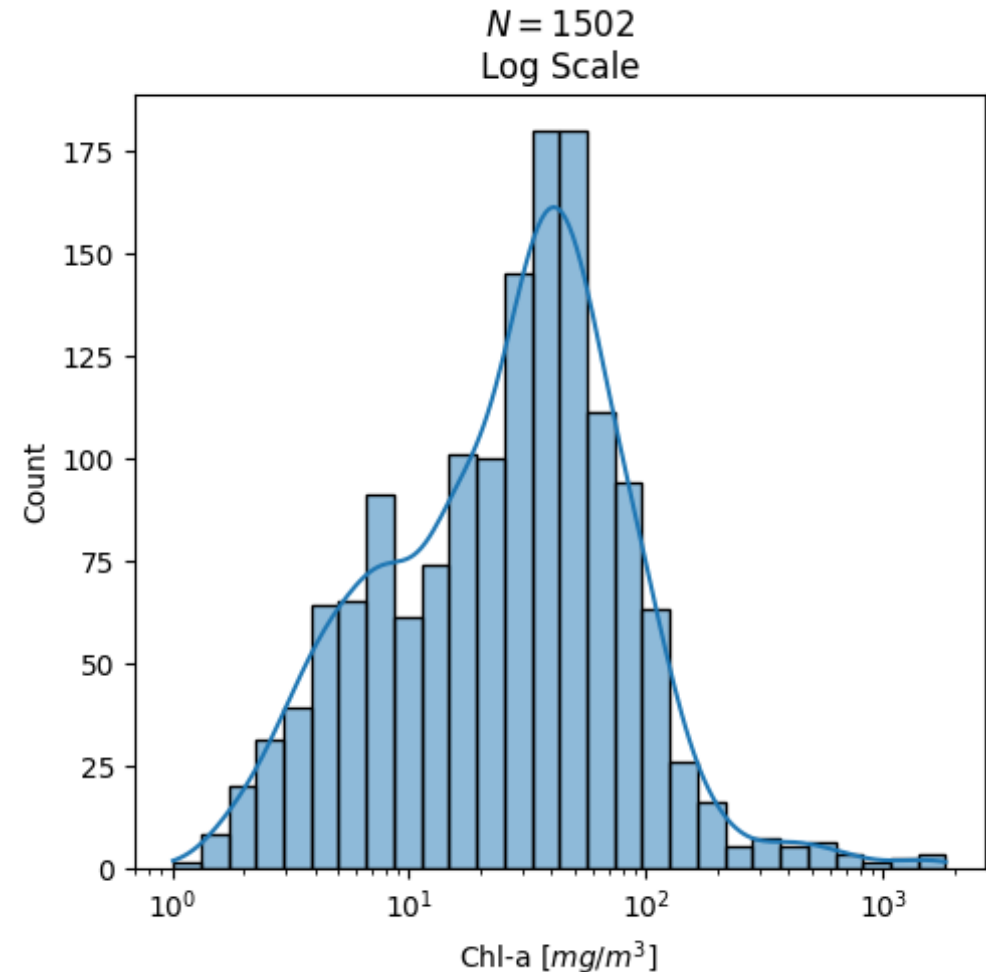
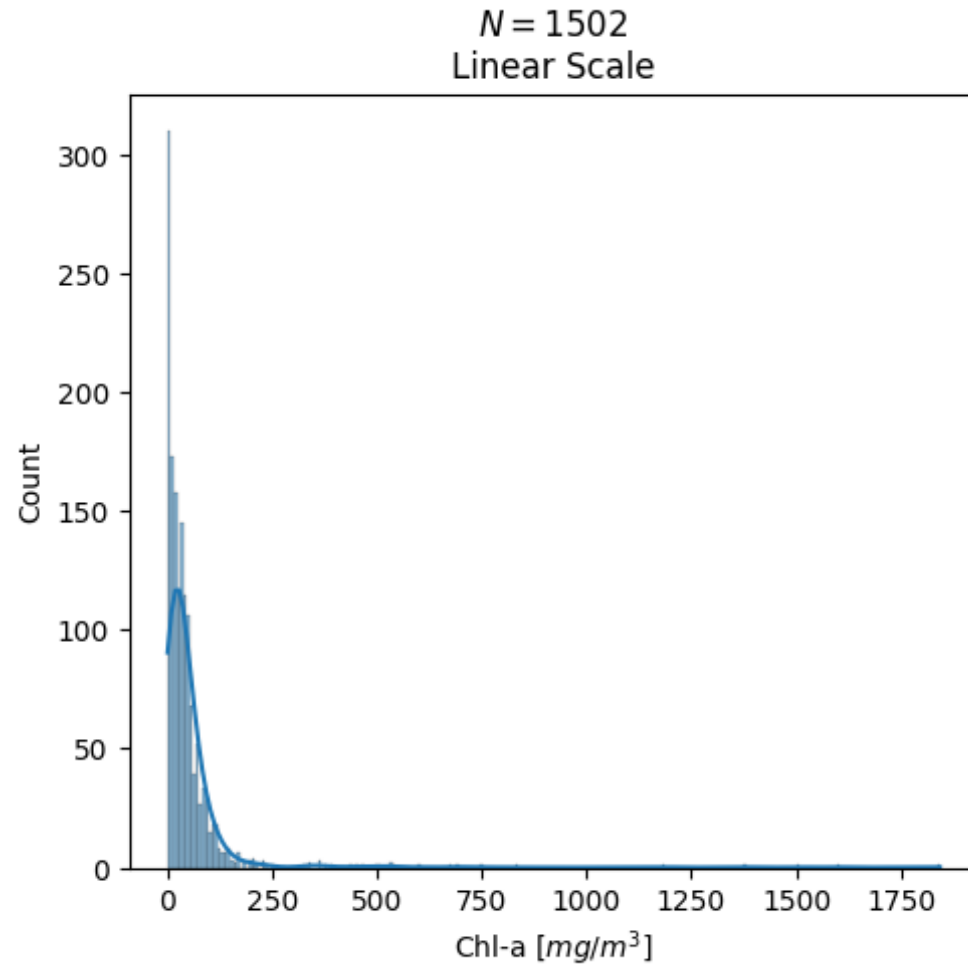
To-do  [get to coding!]

- Follow me on the “01_EDA” notebook

Interpreting Distribution of Target Variable

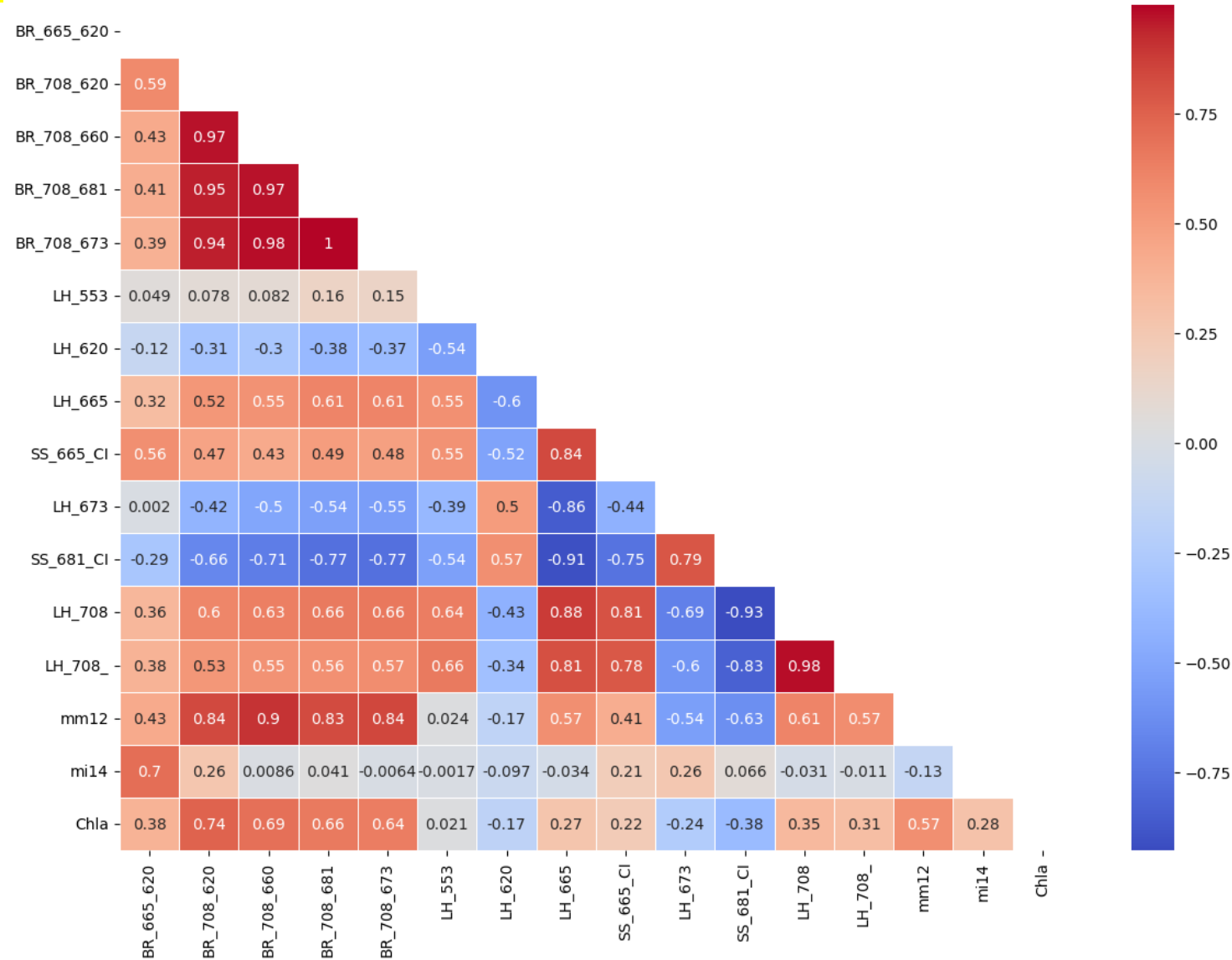
- **Bell Curve/Gaussian**
- **Additive distribution**
 - **Eg: Height of humans**
- **Log-Normal, Skewed-Linear**
- **Multiplicative distribution**
 - **Eg: Income**

Interpreting Distribution of Target Variable



To-do [get to coding!]

- Try out the missing parts of the “01_EDA” notebook. Does your heatmap look like this?
- Try to answer the questions, share what you think with me and others in the class
- When you’re ready, follow instructions on slide “Basic Git commands to Remember” and try to push your changes to your repository



5-min breather

- Reflect on what we did about
- Does (most of) it make sense, does everything work okay?
- Do you have any questions?
- Help each other out!

To-do  [get to coding!]

- Follow me on the “02_ML_modeling” notebook
- Feel free to keep the previous session slides open on the side

To-do [get to coding!]

- Did you try changing the “random_state” during data split? What did you observe?
- Why do you think **ML models** were better than **linear models**?
- Try to answer the questions, share what you think with me and others in the class
- When you're ready, follow instructions on slide “Basic Git commands to Remember” and try to push your changes to your repository

5-min breather

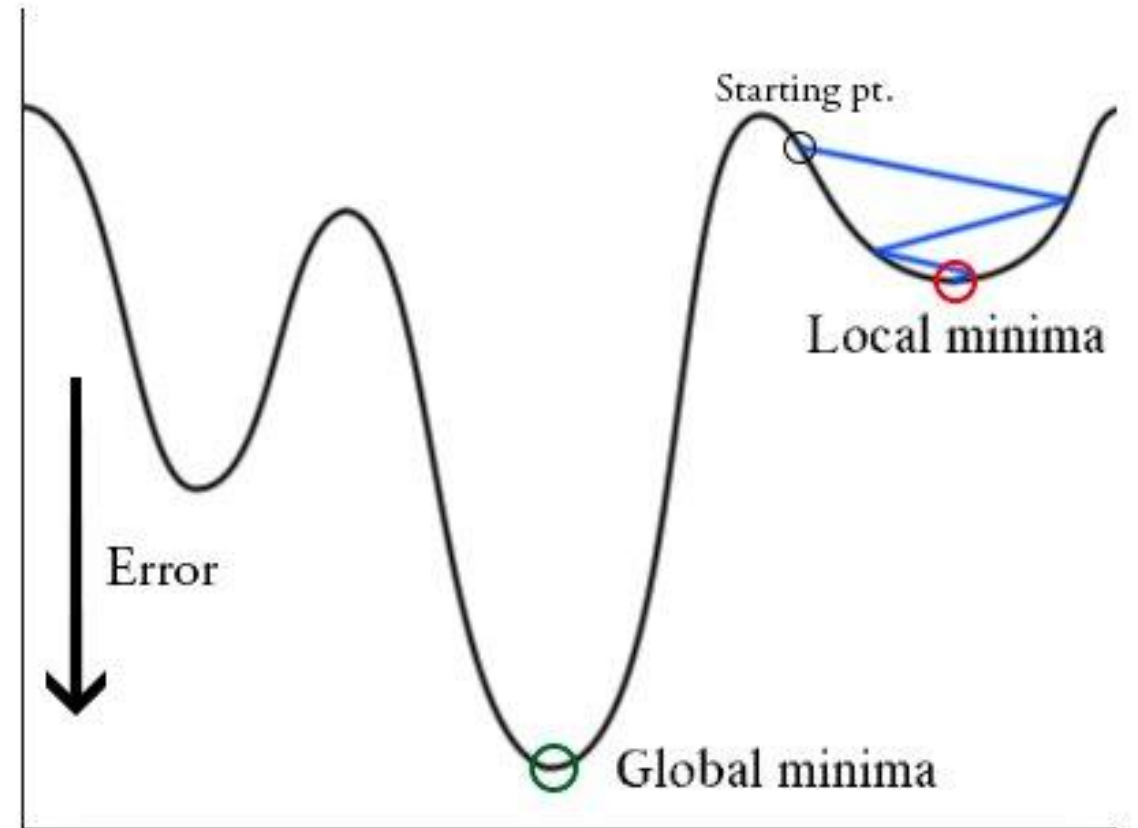
- Reflect on what we did about
- Does (most of) it make sense, does everything work okay?
- Do you have any questions?
- Help each other out!

Transformation, scaling and normalization of Data

- **Data Transformation:** Changing the *shape* of the distribution.
 - *Example: Log-Transforming* Chlorophyll-a. We do this because bio-optical data is usually log-normally distributed (spanning orders of magnitude). This makes the data look more "bell-shaped" (Gaussian), which helps many statistical models.
- **Scaling (Min-Max):** Squishing data into a specific range, usually [0,1].
 - Formula: $X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$
 - *Effect:* It preserves the relative distribution but forces everything into a box.
- **Normalization (Standardization / Z-score):** Centering data around 0 with a standard deviation of 1.
 - Formula: $X_{\text{new}} = (X - \text{mean}) / \text{sd}$
 - *Effect:* It tells you how many "standard deviations" a value is from the mean. It handles outliers slightly better than Min-Max scaling.

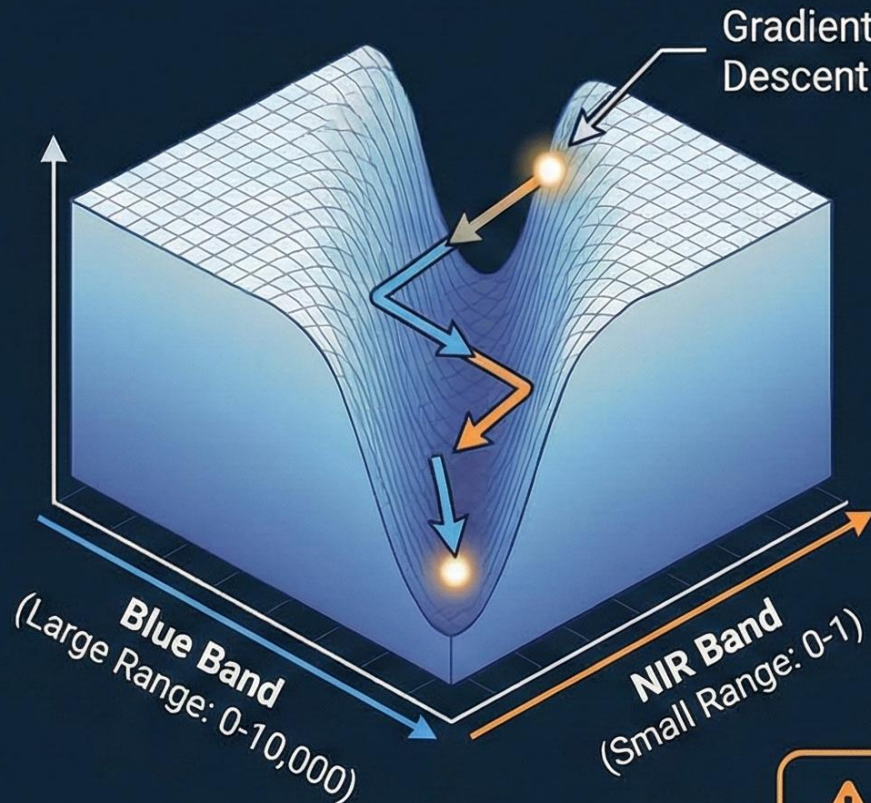
Why do we need these, though?

- For ML models (tree-based), we're okay. They look at the original scale of data and determine the conditions better.
- For NNs (or SVMs), recall – they are universal approximators. Their job is to minimize the loss function.
 - Even though everything is in Rrs, we still have magnitude bias.
 - This can cause NN to get stuck in a local minima, or take longer to train!



Why Neural Networks Need Scaling: The "Sensitive" Ones

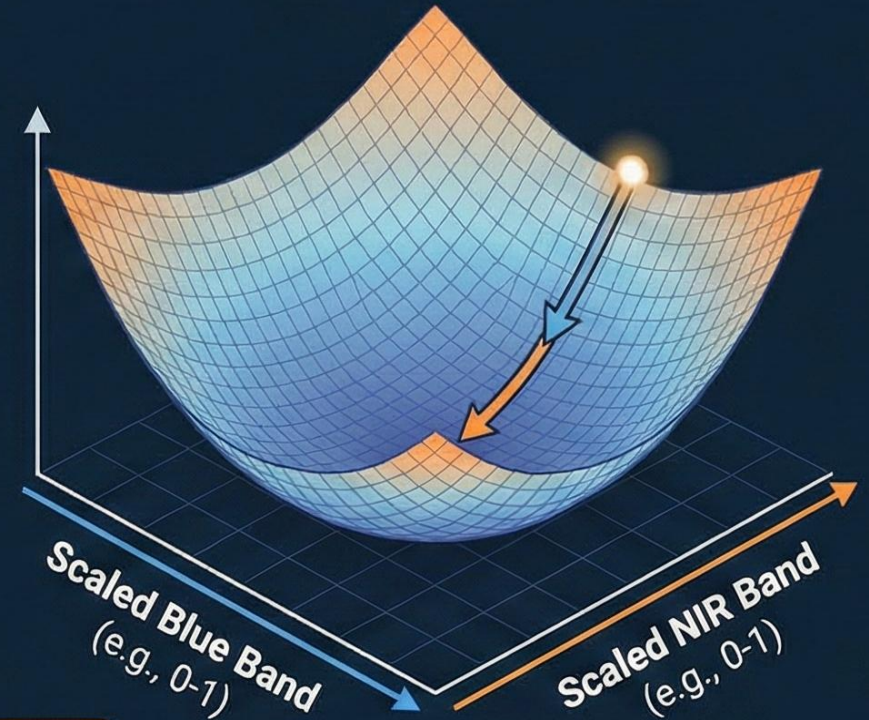
Before Scaling: The Long, Thin Canyon
(Unscaled Variables)



Optimizer bounces along steep walls (large variable), struggles to move down valley (small variable).

Scaling
(e.g., Min-Max,
Standardization)

After Scaling: The Round, Smooth Bowl
(Scaled Variables)



Optimizer walks straight to the bottom quickly and efficiently.

⚠ VERDICT: The Consequence

If you don't scale inputs for a Neural Network, it might **fail to converge**, or it will take **10x longer** to train.

Important caution for Scaling and Splitting

- Always scale your data after you have split your data into train and test.
- Do not scale first, and then split your data.
- Why?
 - Scaling first, you are using the “min” and “max” from your entire dataset.
 - Splitting first, you are using “min” and “max” only of your training data.
 - If you scale first, part of your testing data leaks into your training data, compromising the integrity of your model. The model will inherently do better on test data in this case, because it has already seen the statistics of the test data.

To-do  [get to coding!]

- Follow me on the “03_DL_modeling” notebook
- Feel free to keep the previous session slides open on the side

5-min breather

- Reflect on what we did about
- Does (most of) it make sense, does everything work okay?
- Do you have any questions?
- Help each other out!

To-do  [get to coding!]

- Follow me on the “04_inference_on_satellite_images” notebook
- Feel free to keep the previous session slides open on the side

Few things to remember!

- Don't be scared of coding!
- Grasp the concept, tools and technologies will always keep changing
- Whenever learning something new, be kind to each other and help each other out! 😊
- You all have great potential, good luck ahead, and like my advisor said to me many years ago, “*Do good science!*”

Thank you for listening!

- I hope you all found these sessions useful, and had fun! 😊
- Please feel free to ask any questions/comments/feedback you may have!
- If you have any feedback on my teaching, or content in general, please reach out – I would very much appreciate it!
- You can reach me at:
chintanmaniyar@gmail.com OR chintanmaniyar@uga.edu
- Or LinkedIn:

